```
import pandas as pd
```

```
df=pd.read_csv('/content/healthcare_data_cleaning_dataset.csv')
```

```
df
```

	Patient_ID	Age	Gender	City	Diagnosis	Hospital_Visits	Treatment_Cost	Insurance_Coverage	Admission_Date
0	17270	35.0	Male	Bangalore	Hypertension	13	41010.0	1	2023-11-30
1	10860	21.0	Female	Hyderabad	Flu	11	12194.0	1	2023-02-23
2	15390	77.0	Female	Bangalore	Asthma	2	45086.0	0	2023-03-14
3	15191	79.0	Female	Mumbai	Asthma	13	40842.0	0	2023-08-01
4	15734	60.0	Female	Delhi	Asthma	1	9873.0	1	2023-06-20
...	...	...	...	...	...	...	...	...	...
5095	11764	NaN	Female	Mumbai	COVID-19	15	NaN	0	2023-09-22
5096	17597	NaN	Female	Chennai	Asthma	2	NaN	0	2023-06-26
5097	19171	NaN	Female	Mumbai	Flu	1	NaN	1	2023-12-31
5098	13854	NaN	Female	Bangalore	Flu	17	NaN	0	2023-01-18
5099	14859	NaN	Male	Delhi	Hypertension	19	NaN	0	2023-08-28

5100 rows × 9 columns

Q1. Missing Data Identification

Scenario:

The hospital suspects incomplete patient records.

Task:

Identify missing values in each column

Calculate percentage of missing data

```
df_missing= df.isnull().sum()/len(df)*100
print(df_missing)
```

```
Patient_ID            0.000000
Age                  11.764706
Gender                0.000000
City                  0.000000
Diagnosis             0.000000
Hospital_Visits       0.000000
Treatment_Cost       11.627451
Insurance_Coverage    0.000000
Admission_Date        0.000000
dtype: float64
```

```
df.isnull()
```

	Patient_ID	Age	Gender	City	Diagnosis	Hospital_Visits	Treatment_Cost	Insurance_Coverage	Admission_Date
0	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...
5095	False	True	False	False	False	False	True	False	False
5096	False	True	False	False	False	False	True	False	False
5097	False	True	False	False	False	False	True	False	False
5098	False	True	False	False	False	False	True	False	False
5099	False	True	False	False	False	False	True	False	False

5100 rows × 9 columns

Q2. Handling Missing Age

Scenario:

Age is critical for medical analysis, but some

values are missing.

Task:

Replace missing Age values with an appropriate

method

Justify your choice (mean/median)

```
median_age = df['Age'].median()
df['Age']=df['Age'].fillna(median_age, inplace=True)
```

```
/usr/local/lib/python3.13/dist-packages/numpy/lib/_nanfunctions_impl.py:1241: RuntimeWarning: Mean of empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)
/tmp/ipykernel_1846/588995492.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through ch
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]


  df['Age']=df['Age'].fillna(median_age, inplace=True)
/tmp/ipykernel_1846/588995492.py:2: FutureWarning: Downcasting object dtype arrays on .fillna, .ffill, .bfill is deprecated
  df['Age']=df['Age'].fillna(median_age, inplace=True)
```

Q3. Handling Missing Treatment Cost

Scenario:

Treatment cost is highly skewed due to expensive

treatments.

Task:

Handle missing Treatment_Cost values

Choose the correct imputation method and explain why

```
median_cost = df['Treatment_Cost'].median()
df['Treatment_Cost'] = df['Treatment_Cost'].fillna(median_cost)
```

```
median_cost
```

```
26920.151157458764
```

Q4. Duplicate Patient Records

Scenario:

Some patient records were entered multiple times.

Task:

Identify duplicate rows

Remove duplicates

Compare dataset size before and after

```
print("Duplicate rows:", df.duplicated().sum())
before = len(df)
df = df.drop_duplicates()
after = len(df)
print("Before:", before, "After:", after)
```

```
Duplicate rows: 0
Before: 5001 After: 5001
```

Q5. Invalid Age Values (Data Quality Check)

Scenario:

Some patients have unrealistic age values (e.g.,

> 100 or <0).

Task:

Detect such records

Decide whether to remove or correct them

```
invalid_age = df[(df['Age']>100) | (df['Age']<0)]
print("Invalid age records:", len(invalid_age))
```

```
Invalid age records: 0
```

Q6. Outlier Detection (Treatment Cost)

Scenario:

Extreme treatment costs are affecting analysis.

Task:

Detect outliers using IQR method

Display number of outlier

```
Start coding or generate with AI.
```

Q7. Outlier Treatment

Scenario:

The business team wants to retain all records.

Task:

Apply capping (Winsorization) on Treatment_Cost

Use 5th and 95th percentile

```
p5= df['Treatment_Cost'].quantile(0.05)
p95= df['Treatment_Cost'].quantile(0.95)
df['Treatment_Cost_capping']=df['Treatment_Cost'].clip(lower= p5,upper= p95)
```

```
df['Treatment_Cost_capping']
```

	Treatment_Cost_capping
0	0.95
1	0.95
2	0.95
3	0.95
4	0.95
...	...
4996	0.95
4997	0.95
4998	0.95
4999	0.95
5095	0.95

5001 rows × 1 columns

dtype: float64

Double-click (or enter) to edit

Q8. Transformation

Scenario:

Treatment cost is highly skewed

Task:

Apply log transformation

Create a new column

Compare before vs after distribution

```
import numpy as np
df['Treatment_Cost_Log'] = np.log1p(df['Treatment_Cost'])
print("Skew before log:", df['Treatment_Cost'].skew())
print("Skew after log:", df['Treatment_Cost_Log'].skew())
```

```
Skew before log: 4.485699649227538
Skew after log: -1.1056582956857453
```

Q9. Time-Based Missing Handling

Scenario:

Admission dates should follow a logical sequence.

Task:

Sort data by Admission_Date

Apply forward fill or backward fill where

appropriate

Justify your choice

```
df = df.sort_values(by='Admission_Date').reset_index(drop=True)
df['Age'] = df['Age'].ffill().bfill()
df['Treatment_Cost'] = df['Treatment_Cost'].ffill().bfill()

#No missing Admission_Date values were found in this dataset (0 missing), so no fill was actually needed here.
```